TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

# Simple Assembly Line Balancing Problem

## Progetto per Algoritmi e Modelli per l'Ottimizzazione Discreta

Emanuela Gallinelli
0386515

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

# Il Simple Assembly Line Balancing Problem

Introduzione, Tipologie, Formulazione, Esempio, Sfide

# Introduzione

- Il **SALBP** (Simple Assembly Line Balancing Problem) è un **problema di ottimizzazione combinatoria**: consiste nel bilanciare il lavoro lungo una linea di montaggio.
- Un insieme di operazioni elementari (task), ciascuna con il suo tempo, va assegnato a una serie di stazioni sequenziali, **rispettando i vincoli di precedenza tra i task**.
- È "***Simple***" perché il modello è di base: un solo prodotto, linea seriale, tempi deterministici e nessun vincolo oltre alle precedenze.
- Si applica in settori come l'automotive, l'elettronica e gli elettrodomestici.



# Tipologie di SALBP

**SALBP-1 (Minimizzazione delle stazioni)**

- **Obiettivo:** Minimizzare il numero totale di stazioni di lavoro ($m$).
- **Dato fisso:** Il tempo di ciclo ($c$).

**SALBP-2 (Minimizzazione del tempo di ciclo)**

- **Obiettivo:** Minimizzare il tempo di ciclo ($c$)
- **Dato fisso:** Il numero di stazioni ($m$).

**SALBP-E (Massimizzazione dell'efficienza)**

- **Obiettivo:** Minimizzare il tempo totale a disposizione (m*c)
- **Dati variabili:** il numero di stazioni e il tempo di ciclo sono incognite.

**SALBP-F (Verifica di fattibilità)**

- **Obiettivo:** Determinare se esiste almeno un'assegnazione di task valida
- **Dati fissi:** il numero di stazioni e il tempo di ciclo sono imposti come vincoli rigidi.

# Formulazione matematica

## Parametri e Variabili

- $\boldsymbol{i, j} \in \boldsymbol{V}$: i task da eseguire, $t_i$ è il tempo di esecuzione del task $i$
- $\boldsymbol{s} \in \{\boldsymbol{1}, \dots, \boldsymbol{m}\}$: le stazioni di lavoro
- $\boldsymbol{A}$: insieme delle precedenze; $(i, j) \in A$ significa che $i$ deve essere eseguito prima di $j$
- $\boldsymbol{x_{is}} \in \{\boldsymbol{0}, \boldsymbol{1}\}$**:** Variabile binaria. Vale 1 se il task $i$ è assegnato alla stazione $s$, 0 altrimenti.
- $\boldsymbol{LB \leq c \leq c_{max}}$: tempo di ciclo, ovvero il massimo carico tra le stazioni.
- $\boldsymbol{E_i(c_{max})}$, $\boldsymbol{L_i(c_{max})}$: Finestre di assegnazione (*Earliest* e *Latest station*). Indicano l'intervallo di stazioni possibili per il task $i$ in base ai tempi e alle precedenze.

## Funzione Obiettivo e Vincoli

- **Funzione Obiettivo:** $\min\ c$
- **Vincolo di assegnazione:**

$$\sum_{s=E_i}^{L_i} x_{is} = 1 \quad \forall i \in V$$

- **Vincolo di capacità:**

$$\sum_{i} t_i \cdot x_{is} \leq c \quad \forall s \in \{1, \dots, m\}$$

- **Vincolo di precedenza:**

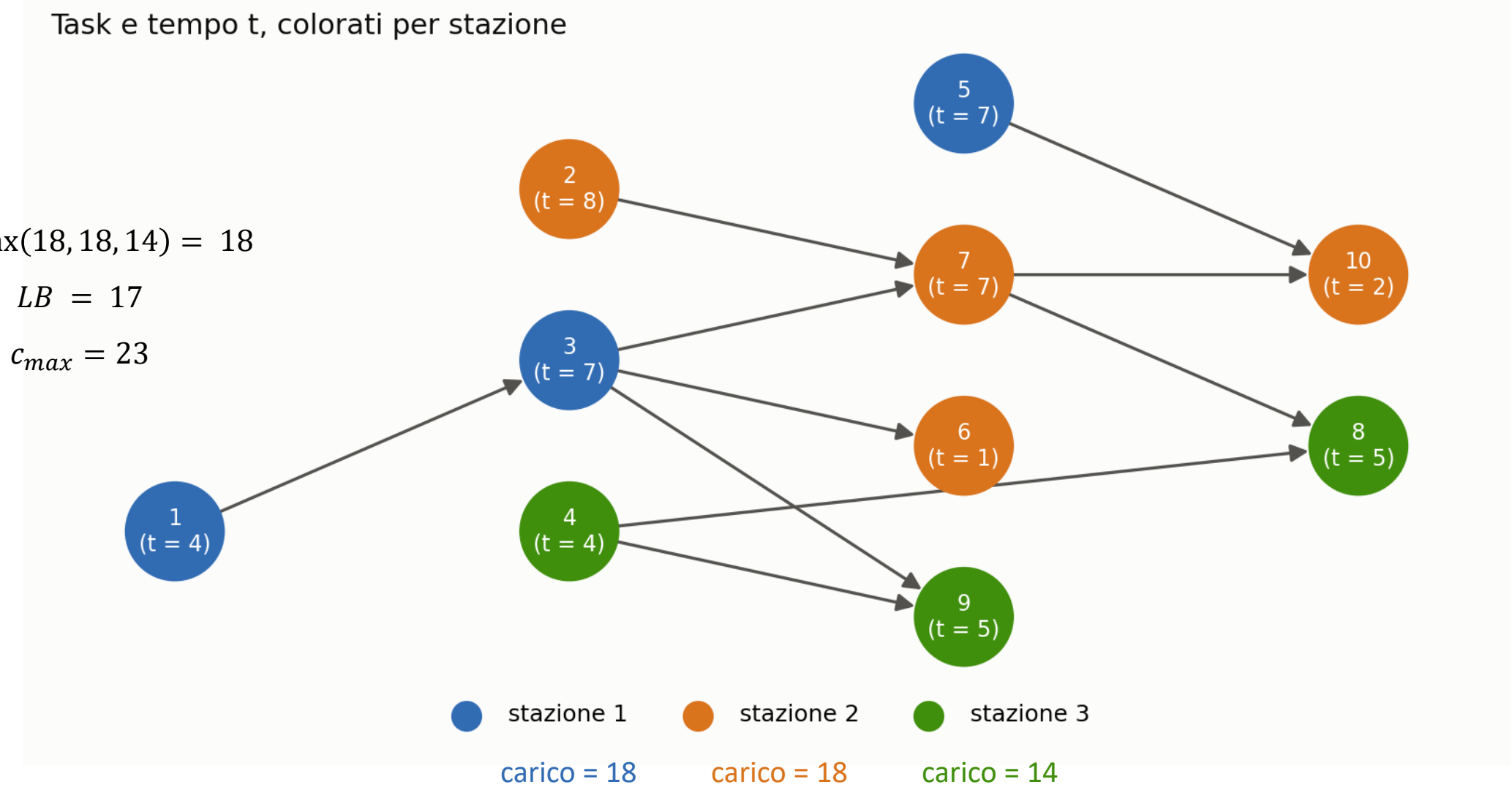
$$stazione(i) \leq stazione(j) \quad \forall (i, j) \in A$$

# Esempio

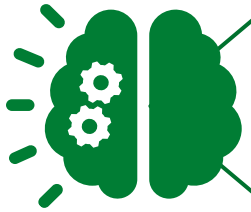
$c = \max(18, 18, 14) = 18$

$LB = 17$

$c_{max} = 23$



# Sfide



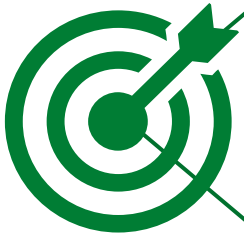
### Complessità computazionale

Il problema di fattibilità (SALBP-F) è NP-completo, quindi **SALBP-2 è NP-HARD** e le possibili assegnazioni dei task alle stazioni crescono in modo esponenziale con *n*



### Difficoltà variabile delle istanze

A parità di numero di task, la difficoltà dipende da densità delle precedenze e numero di stazioni: istanze della stessa dimensione possono essere banali o molto difficili.



### Dimostrare l'ottimalità

Trovare una buona soluzione è spesso più facile che dimostrare che sia ottima: servono lower bound stretti sul tempo ciclo.

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

# Definizione dei Bound

Lower-Bound, Finestre di Stazione, Teste e code ricorsive

# Lower Bound sul tempo ciclo

Il lower bound è il **massimo di tre bound combinatori** (Scholl & Becker, 2006), calcolati in frazioni di secondo.

## LC1 (Carico medio)

$$c \geq \max\left\{\left\lceil \sum t_i / m \right\rceil, t_{max}\right\}$$

*Esempio:* $max\left\{\left\lceil \frac{50}{3} \right\rceil, 8\right\} = 17$

## LC2 (Task lunghi)

**Tra i $k \cdot m + 1$ task più lunghi, almeno $k + 1$ finiscono nella stessa stazione**

*Esempio*: $k = 1$: tra i 4 più lunghi (8, 7, 7, 7) due stanno insieme, considerando tra questi i $k + 1$ task più corti

$$c \geq 7 + 7 = 14$$

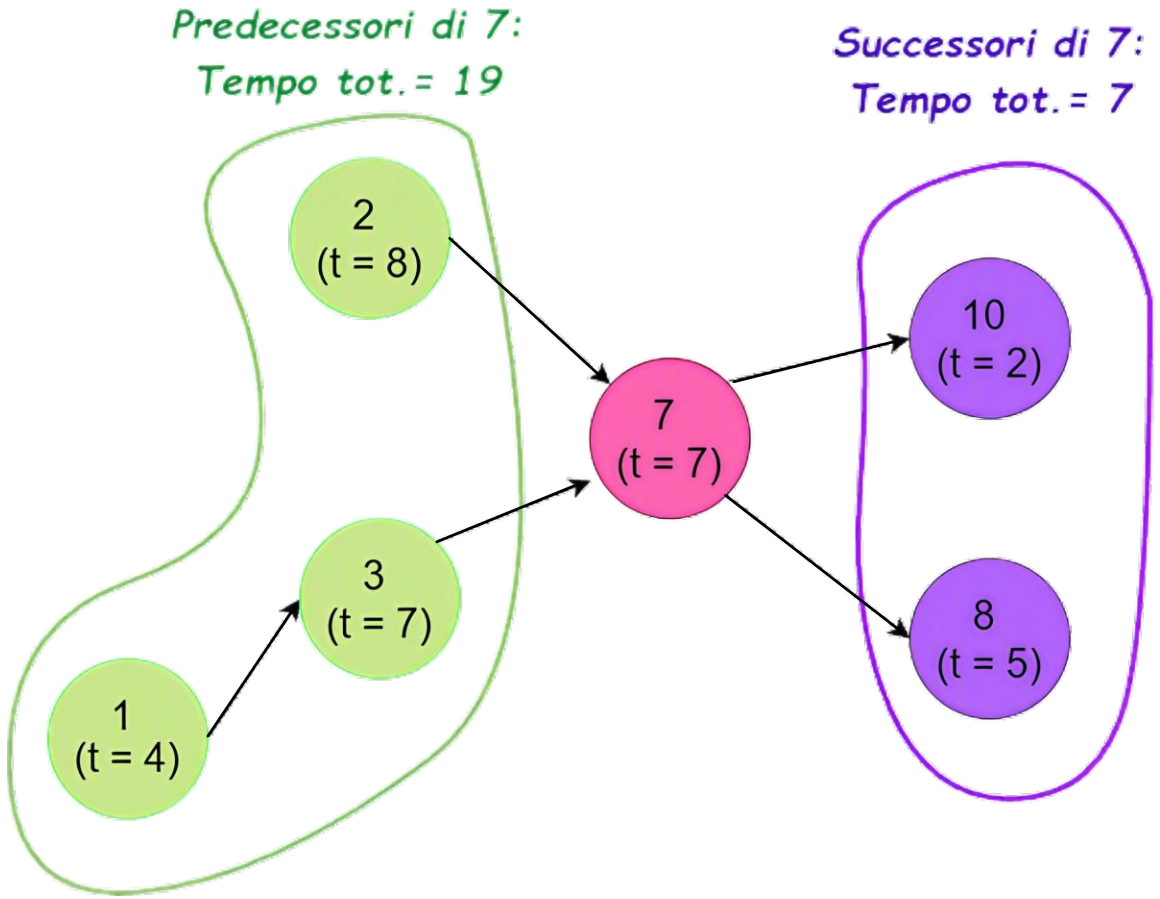
## LC3 (Finestre di stazione)

**Se un task non ha stazioni ammissibili, c è irraggiungibile**

*Esempio*: Partendo da 17 (LC1), tutte le finestre sono non vuote, quindi LC3 = 17

# Finestre di stazione (LC3)

- Per un tempo ciclo c, ogni task può stare solo in un intervallo di stazioni $[\boldsymbol{E}_i(\boldsymbol{c}), \boldsymbol{L}_i(\boldsymbol{c})]$.
- $\boldsymbol{E}_i(\boldsymbol{c}) = \lceil (\boldsymbol{t}_i + \boldsymbol{tempi\ predecessori}) / \boldsymbol{c} \rceil$
- $\boldsymbol{L}_i(\boldsymbol{c}) = \boldsymbol{m} + \boldsymbol{1} - \lceil (\boldsymbol{t}_i + \boldsymbol{tempi\ successori}) / \boldsymbol{c} \rceil$
- *Esempio (m = 3, c = 12, valore di prova): il task 7 ha tempo 7; i suoi predecessori sommano 19, i successori 7.*
  - $E_7\,(12) = \left\lceil \frac{7+19}{12} \right\rceil = \left\lceil \frac{26}{12} \right\rceil = 3$ → Deve stare nella stazione 3 o dopo
  - $L_7(12) = 3 + 1 - \left\lceil \frac{7+7}{12} \right\rceil = 4 - 2 = 2$ → Deve stare nella stazione 2 o prima
  - Finestra [3,2] → Finestra vuota → c = 12 impossibile




*Sottografo dell'esempio: solo i task legati al task 7*

# Teste e code ricorsive

| | |
|---|---|
| **Testa di un task** | Lavoro che deve essere già stato fatto prima che il task possa iniziare. |
| **Coda di un task** | Lavoro che resta da fare dopo che il task è finito. |
| **Semplici:** | Somma dei tempi di tutti i predecessori (testa) o di tutti i successori (coda), come se il lavoro si potesse dividere liberamente tra le stazioni. |
| **Ricorsive (Johnson, 1988):** | Si calcola a partire dalle teste (o code) dei predecessori (o successori) e tiene conto che i task non si dividono. Se un task non entra nello spazio rimasto in una stazione, inizia nella successiva e lo spazio vuoto si aggiunge alla testa. È più forte con catene lunghe di precedenze.<br>*Nella campagna: la fase 2 ha alzato il bound in 113 istanze (60 uniformi, 53 bimodali), tutte a OS 0,9* |



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

# PLI (Programmazione Lineare Intera)

Patterson & Albracht, Ritt & Costa

# Patterson & Albracht

- Le precedenze si scrivono confrontando direttamente il **numero di stazione** dei due task: se $i$ precede $j$, la stazione di $i$ non può superare quella di $j$.
- **Vincolo di precedenza**, per ogni arco $(i, j)$:

$$\sum_{s=E_i}^{L_i} s \cdot x_{is} \leq \sum_{s=E_j}^{L_j} s \cdot x_{js}$$

- **Un solo vincolo per arco,** gli archi sono ridotti con la **riduzione transitiva** in entrambi i modelli.
- **Rilassamento lineare debole**: la somma pesata con $s$ aggrega l'informazione, quindi con variabili frazionarie il vincolo è facile da soddisfare e il bound è poco stretto.

# Ritt & Costa (1)

- Invece di confrontare i numeri di stazione, si impone che per ogni stazione $k$ il task $i$ sia stato assegnato entro $k$ *almeno quanto* il suo successore $j$: $j$ non può stare in una stazione precedente a quella di $i$.

- **Vincolo di precedenza**, per ogni arco $(i, j)$ e ogni stazione $k$:

$$\sum_{s \le k} x_{is} \geq \sum_{s \le k} x_{js}$$

- **Tagli sulle finestre di stazione:** Il tempo ciclo è modellato con variabili binarie

$$r_t \ (t = c_{min}, \dots, c_{max}), \qquad con \ \sum_t r_t = 1 \ e \ c = \sum_t t \cdot r_t$$

# Ritt & Costa (2)

- Le finestre $\boldsymbol{E_i(t)}$, $\boldsymbol{L_i(t)}$ dipendono da $t$: se $t$ è piccolo le finestre si restringono, e i tagli impediscono assegnamenti fuori finestra:

$$\sum_{u \le e} x_{iu} \le 1 - \sum_{t: e < E_i(t)} r_t \quad \forall i \in V, E_i(c_{max}) \le e < E_i(c_{min})$$

$$\sum_{u \ge l} x_{iu} \le 1 - \sum_{t: L_i(t) < l} r_t \quad \forall i \in V, L_i(c_{min}) < l \le L_i(c_{max})$$

# Confronto tra i due modelli PLI

| | Patterson & Albracht | Ritt & Costa |
|---|---|---|
| **Vincoli di precedenza** | 1 per arco: $\sum_s s \cdot x_{is} \leq \sum_s s \cdot x_{js}$ | 1 per arco e per stazione k: $\sum_{s \leq k} x_{is} \geq \sum_{s \leq k} x_{js}$ |
| **Numero di vincoli** | 1 per arco<br>(media a 200 task: circa 740) | Fino a m per arco<br>(media a 200 task: 11.900 con tempi uniformi,<br>13.600 con bimodali) |
| **Finestre di stazione** | Statiche: $E_i(c_{max})$, $L_i(c_{max})$ | Statiche + tagli dinamici in funzione di c |
| **Variabili aggiuntive** | Nessuna | $r_t$ (tempo ciclo) |
| **Rilassamento lineare** | Più debole | Più forte (domina PA) |

***Domanda sperimentale:*** *la forza del rilassamento compensa la maggiore dimensione del modello?*

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

# Euristiche

Euristica dei Gruppi vs Euristica di Hoffmann adattata a SALBP-2

# Euristica dei gruppi

### Idea

Raggruppare i task con precedenze vicine e disporli lungo la linea.

Le **foglie** (task senza successori) fungono esclusivamente da riempitivi.

### Riempimento (con tempo ciclo di prova *c*)

Inserimento dei gruppi per priorità (spezzandoli se non entrano) o uso di foglie per avvicinare il carico a $c$.

Ricerca di $c$ partendo dal Lower Bound (LB) a salire: se servono più di $m$ stazioni, si tenta con $c + 1$.

### Costruzione

**Gruppi a ondate**: ogni radice si unisce ai figli i cui predecessori sono tutti già in un gruppo; gli altri aspettano l'ondata successiva

**Grafo dei gruppi:** priorità per livello (crescente) e costo (decrescente).

### Evoluzione

**Nuova regola:** *riempire ogni stazione il più possibile* riducendo gli spazi vuoti.

Portando la regola all'estremo i gruppi diventano superflui: resta la saturazione delle stazioni, che in letteratura è l'euristica di Hoffmann (1963)

# Euristica di Hoffmann adattata al SALBP-2

Si costruisce la linea una stazione alla volta, scegliendo tra i task disponibili (predecessori già assegnati) l'insieme il cui carico è il più vicino possibile a c senza superarlo.

**Saturazione** = problema di tipo zaino con precedenze, risolto con ricerca in profondità con memoria degli insiemi visitati.

**Schema**:

Per *c* = LB, LB+1, …

→ per *s* = 1…*m* saturare la stazione → se tutti i task sono assegnati, stop.

*Rispetto all'originale (pensato per il SALBP-1): ricerca su c a partire dal LB (Scholl & Becker, 2006) e limite di 20.000 nodi per stazione, raggiunto una sola volta nella campagna*

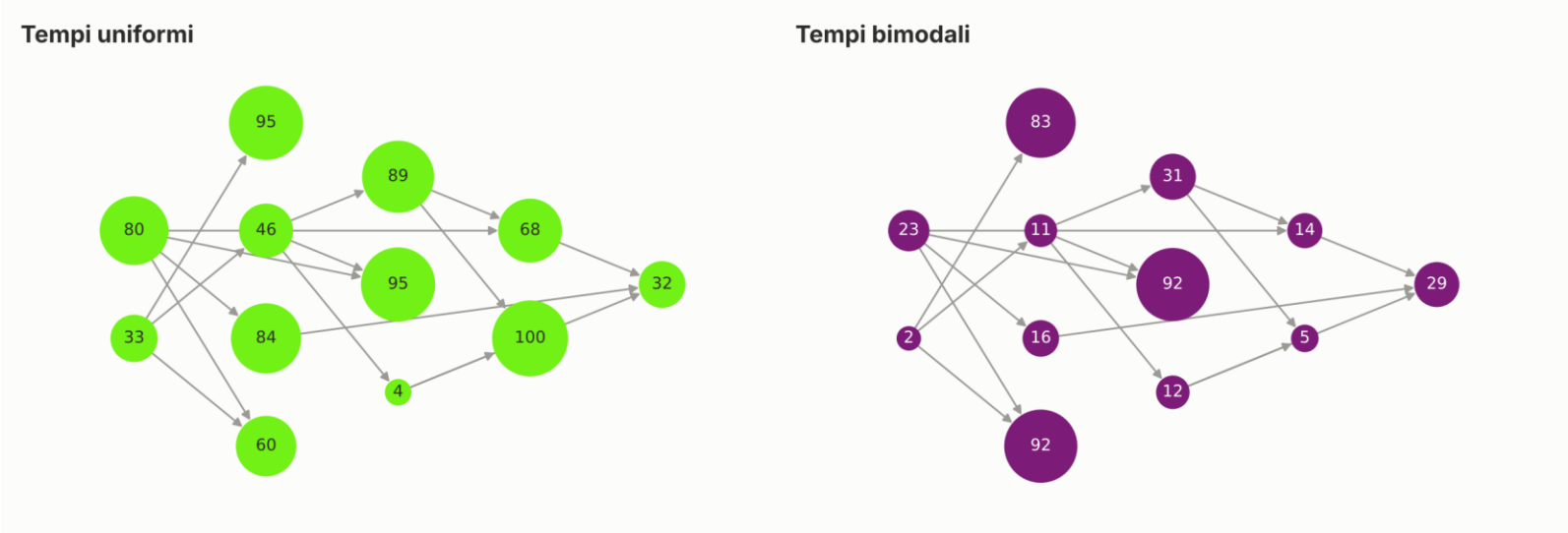
TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

# Istanze

Generazione e disegno sperimentale

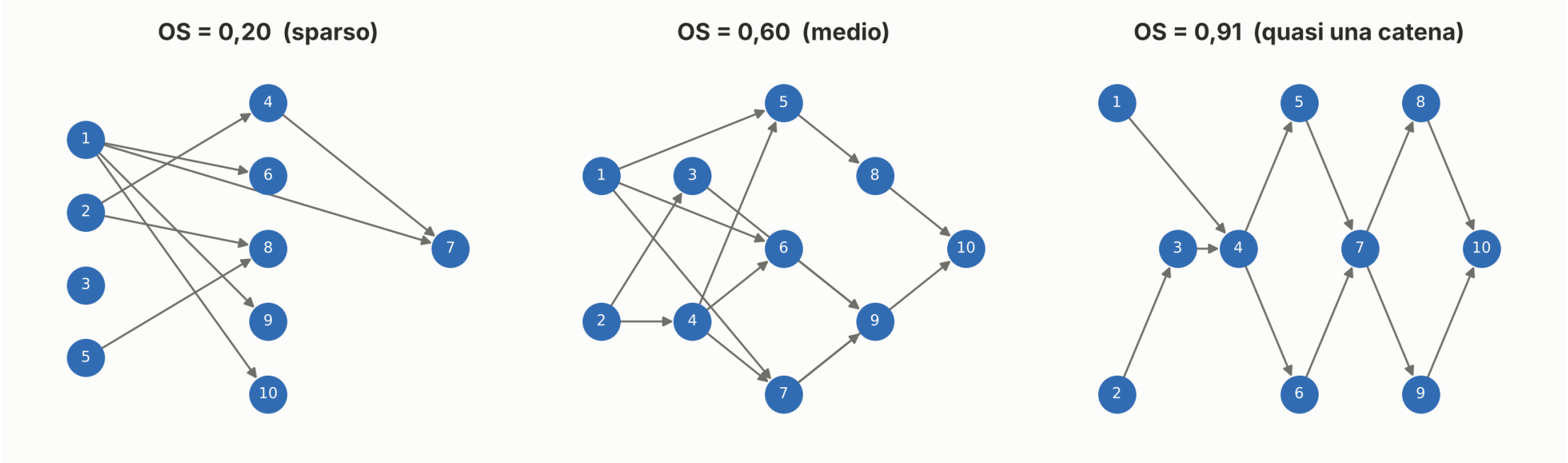
# Generazione delle istanze

- **Order strength (OS)**: frazione delle coppie di task il cui ordine è fissato dalle precedenze, dirette o indirette.
- Il generatore aggiunge archi casuali u → v con u < v e **si ferma quando l'OS raggiunge l'obiettivo**.
- **Tempi dei task**: interi uniformi tra 1 e 100 oppure bimodali (Otto et al., 2013): 80% tra 1 e 33, 20% tra 68 e 100
- **Riproducibilità**: il seed di ogni istanza dipende solo da seed di base, gruppo e indice.




*Con i tempi bimodali i tanti task piccoli riempiono gli spazi lasciati da quelli grandi: le stazioni si riempiono più facilmente e le istanze risultano più facili*

# Order Strength (OS)




**OS → 0**

Ordine libero

**OS → 1**

Unica sequenza

# Disegno sperimentale

| Fattore | Livelli |
|---|---|
| **Numero di task n** | 20, 50, 100, 200 |
| **Order strength** | 0,2 (sparso), 0,6 (medio), 0,9 (denso) |
| **Numero di stazioni m** | n/3 (tante), n/6 (medie), n/10 (poche) |
| **Distribuzione dei tempi** | Uniforme, bimodale |

- Fattoriale completo: 72 gruppi × 25 istanze = 1800 istanze.
- Da Otto, Otto & Scholl (2013): n = 20, 50, 100, i livelli di OS e 25 istanze per gruppo.
- n = 200 è una scelta del progetto, per avere un livello in cui i PLI non chiudono più: vedremo che con tempi uniformi si chiude solo il 27% delle istanze (con tempi bimodali l'81%).
- Numero di stazioni: nel SALBP-2 il tempo ciclo non è un dato, quindi la difficoltà si regola con i task per stazione (circa 3, 6 o 10).
- Tempi bimodali: tanti task brevi e pochi lunghi, come nelle linee reali (Otto et al.)

# Impostazioni degli esperimenti

| | |
|---|---|
| **Solver** | Gurobi, tramite PuLP |
| **Limite di tempo** | 180 secondi per modello e istanza |
| **Thread** | 6 |
| **Esecuzione** | in sequenza, un metodo alla volta |
| **Macchina** | AMD Ryzen 5 5500U, 8 GB di RAM |

- 7200 esecuzioni, circa 48h
- Per ogni PLI si registra **ottimo e tempo**, oppure allo scadere **miglior soluzione e lower bound**.
- Ogni euristica è confrontata con l'ottimo, o con il **miglior lower bound** se l'ottimo non è noto.
- MIPGapAbs (*tolleranza assoluta sul gap*) = 0,999 (*c* è intero: gap < 1 = ottimo)

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

# Risultati

1800 istanze, 4 metodi

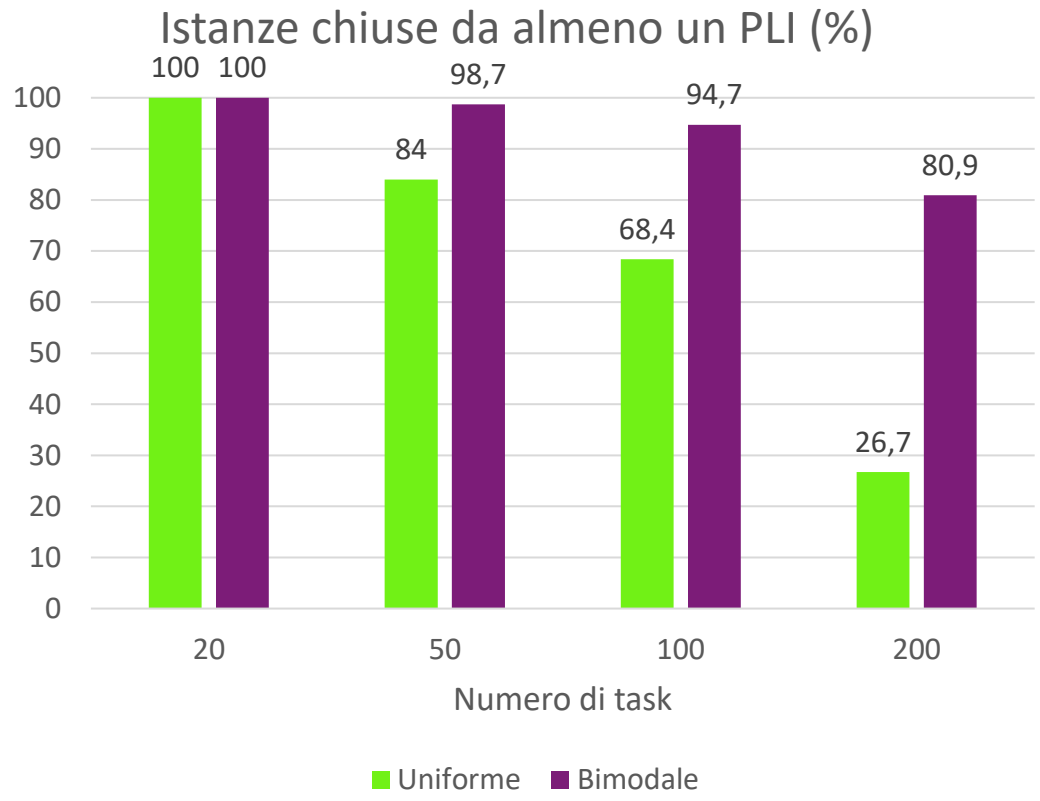
# Panoramica

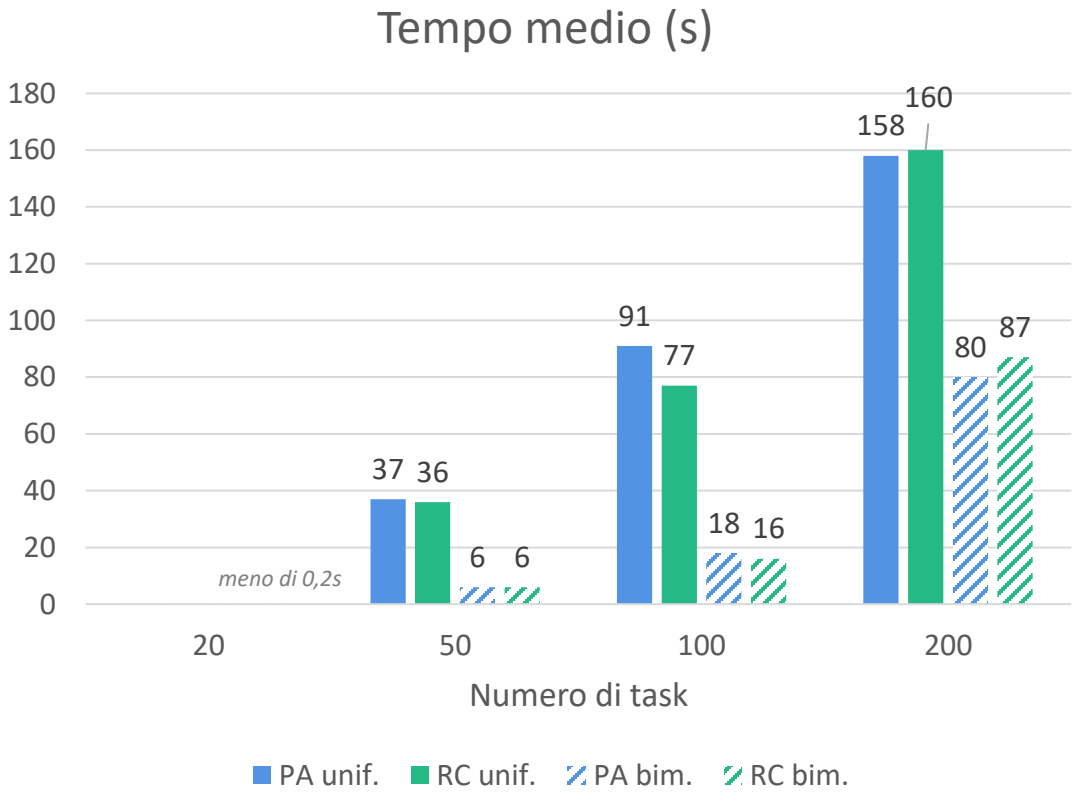
| | PA | RC | Hoffmann | Gruppi |
|---|---|---|---|---|
| **Uniforme: ottimi dimostrati** | 63,9% | 66,7% | 45,0% | 2,8% |
| **Uniforme: ottimo raggiunto*** | 95,1% | 95,7% | 79,5% | 8,8% |
| **Uniforme: tempo medio** | 71,6 s | 68,3 s | 0,11 s | 0,12 s |
| **Bimodale: ottimi dimostrati** | 90,7% | 92,0% | 66,4% | 21,2% |
| **Bimodale: ottimo raggiunto*** | 97,6% | 98,3% | 88,1% | 27,0% |
| **Bimodale: tempo medio** | 25,8 s | 27,4 s | 0,11 s | 0,11 s |

**Calcolato sulle istanze chiuse da almeno un PLI: 628 uniformi e 842 bimodali.*

*Hoffmann trova l'ottimo nell'80% delle istanze chiuse dai PLI con tempi uniformi e nell'88% con tempi bimodali, contro il 95–98% dei PLI, in 0,1 s invece di 25–70 s*

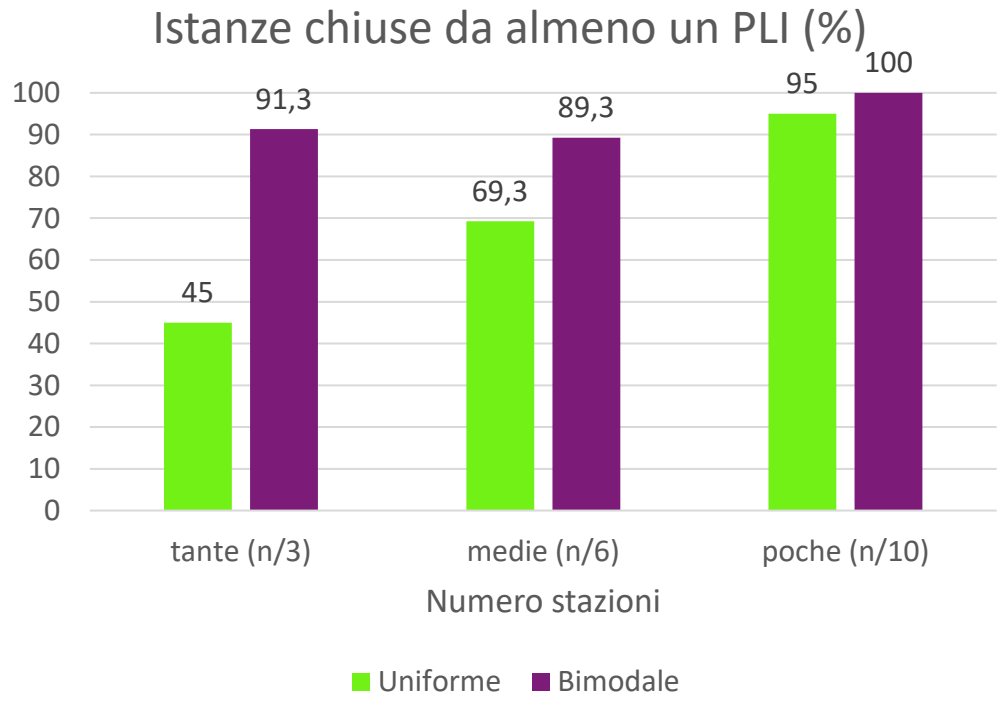
# Chiusura dei PLI e tempi




- *Fino a 20 task i PLI chiudono tutto in meno di un secondo. A 50 task si chiude il 99% con tempi bimodali (circa 6 s) e l'84% con tempi uniformi (circa 37 s).*
- A 200 task con tempi uniformi si chiude solo 1 istanza su 4; con tempi bimodali l'81%

# Cosa rende difficile un'istanza

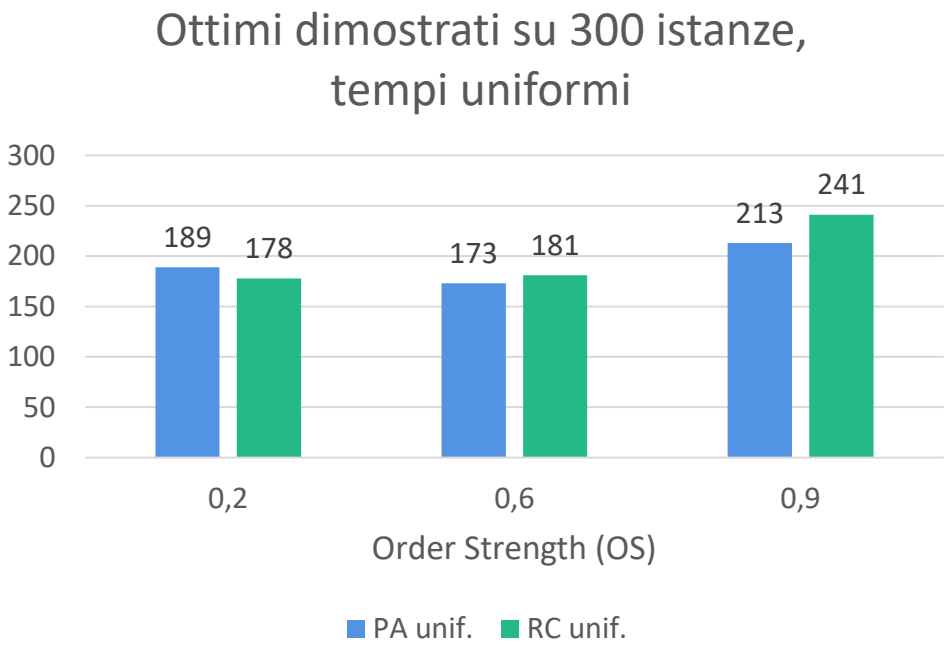


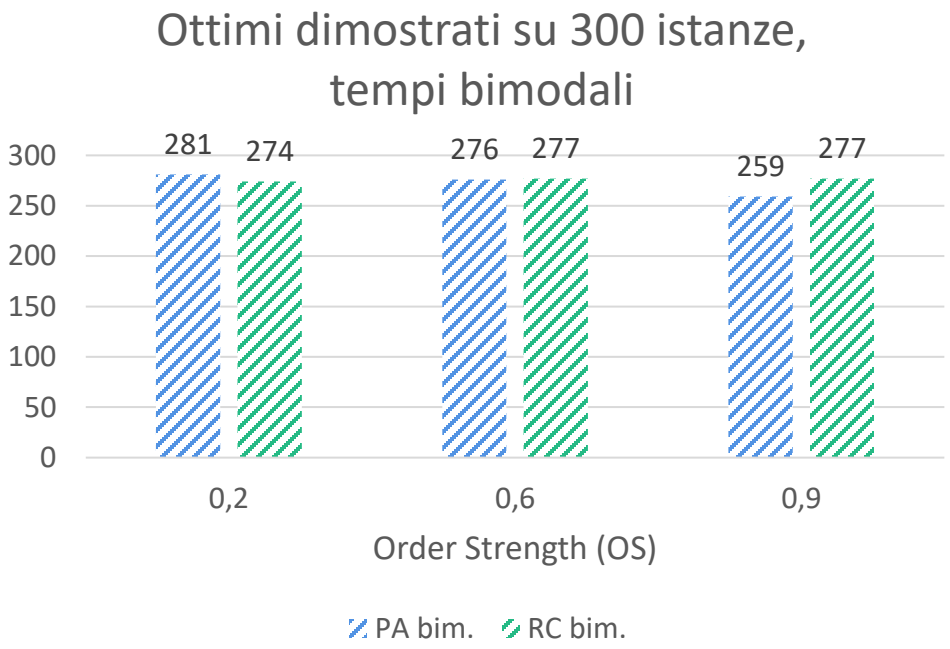

*Con tempi uniformi le istanze dense (OS 0,9) sono le più facili: le finestre si restringono. Con i bimodali l'OS conta poco*

| OS | Uniforme | Bimodale |
|---|---|---|
| **0,2** | 66,3% | 94,3% |
| **0,6** | 62,7% | 94,0% |
| **0,9** | 80,3% | 92,3% |

- *Con poche stazioni ci sono molte combinazioni per riempirle e il lower bound viene raggiunto facilmente.*
- *Con tante stazioni l'impacchettamento è rigido ed è difficile dimostrare l'ottimo.*
- *Con tempi uniformi la difficoltà cresce nettamente con il numero di stazioni: 95% → 69% → 45%.*
- *Con tempi bimodali si chiude quasi tutto; con poche stazioni si arriva al 100%.*

# Patterson&Albracht vs Ritt&Costa




- *Con precedenze dense (OS 0,9) RC dimostra più ottimi (241 contro 213 e 277 contro 259): i vincoli cumulativi e i tagli sulle finestre sono più forti.*
- *Con precedenze sparse (OS 0,2) PA è pari o migliore: ha molti meno vincoli (a 200 task con OS 0,2, in media circa 710 vincoli contro 15.500) e il solver lo esplora più in fretta*
- *Nessuno dei due domina sempre.*

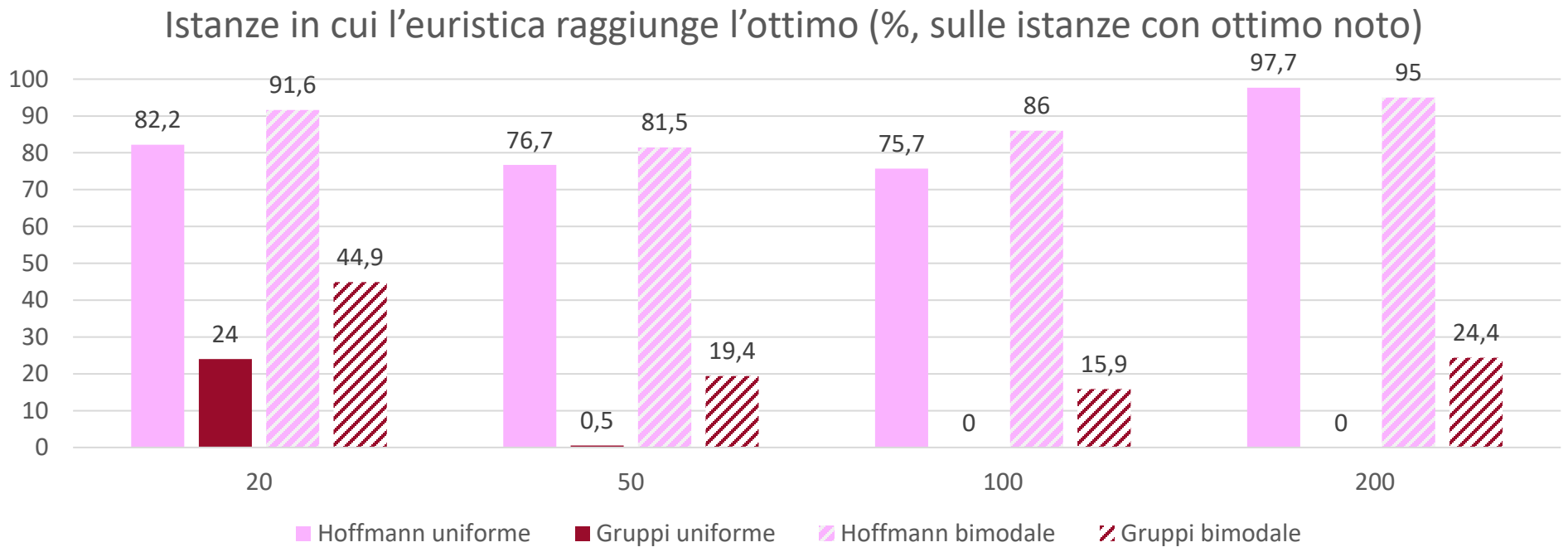
# Dove i modelli differiscono di più

Nella media PA e RC sono vicini. In alcuni gruppi però la differenza è netta *(Nella tabella: % di istanze in cui il modello dimostra l'ottimo)*

| Gruppo | PA | RC |
|---|---|---|
| **Uniforme, 100 task, OS 0,9, tante stazioni (n/3)** | 12% | 84% |
| **Uniforme, 200 task, OS 0,2, poche stazioni (n/10)** | 88% | 44% |
| **Bimodale, 200 task, OS 0,9, stazioni medie (n/6)** | 0% | 28% |

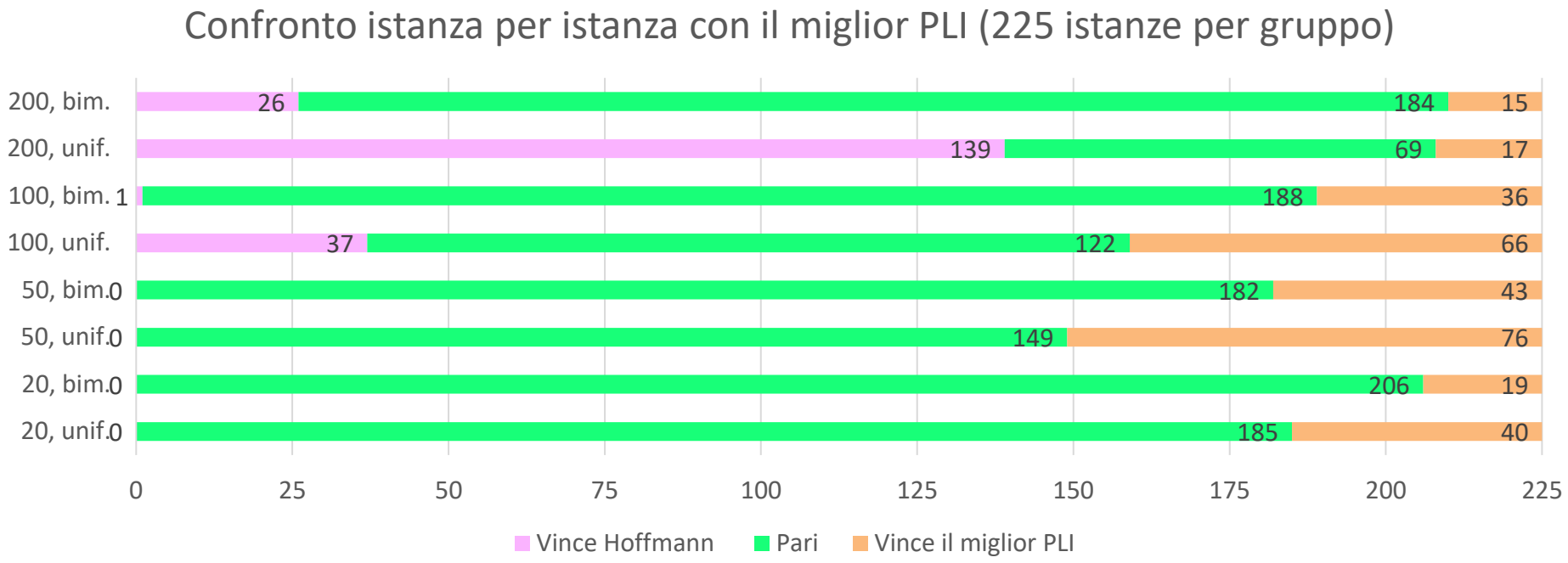
- **Precedenze dense (OS 0,9) → vince RC.** Con molte precedenze i vincoli cumulativi e i tagli sulle finestre escludono molte assegnazioni, e il rilassamento lineare si avvicina all'ottimo.
- **Precedenze sparse con molti task → vince PA.** I vincoli di RC servono a poco, ma ce ne sono molti di più (uno per arco e per stazione). Il modello compatto di PA viene esplorato più rapidamente.
- **Quindi nessun modello domina:** la scelta dipende dalla struttura delle precedenze.

# Hoffmann vs Gruppi



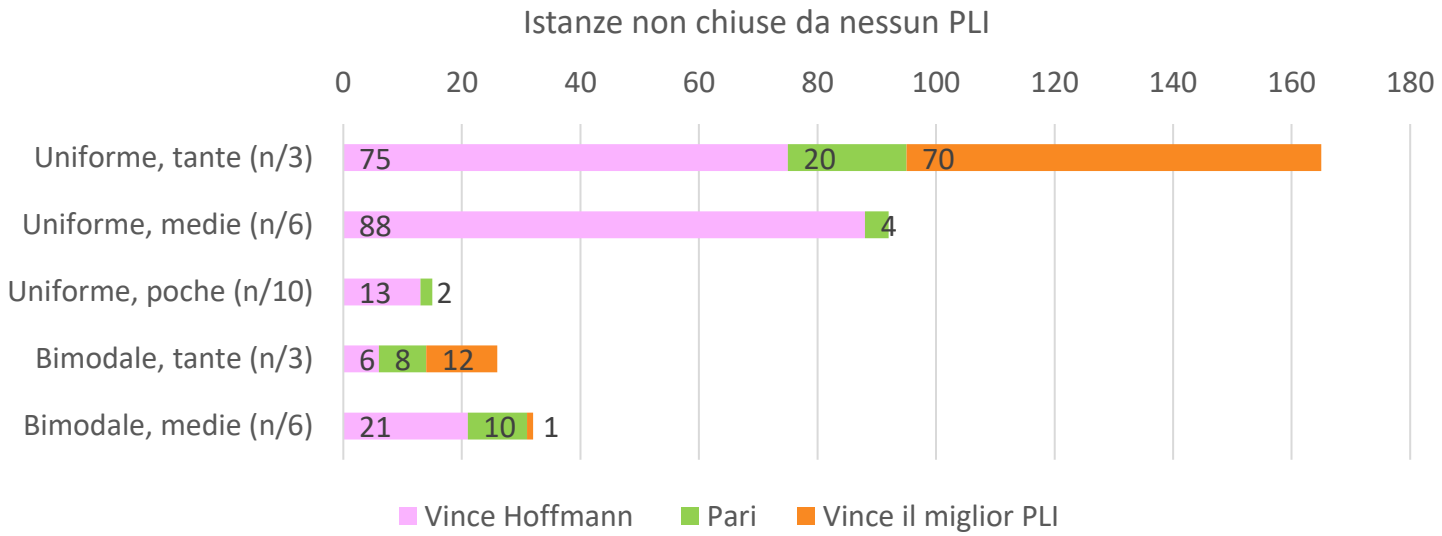

- *Le due euristiche hanno lo stesso ciclo esterno su c, cambia solo come si riempie la stazione.*
- *Hoffmann è migliore in 1504 istanze su 1800, Gruppi in 4 (Wilcoxon* $p < 0{,}001$*)*
- *Scarto medio dal lower bound: Hoffmann 0,7% / 0,2%, Gruppi 3,9% / 2,5% (uniforme / bimodale).*
- *A 200 task l'ottimo è noto soprattutto nelle istanze facili: il valore alto non significa che Hoffmann migliori con n*

# Hoffmann vs PLI




- *Fino a 50 task Hoffmann non batte mai il miglior PLI; a 100 task uniformi vince in 37 casi su 225*
- *A 200 task uniformi vince Hoffmann in 139 istanze su 225: i PLI si fermano a 180 s con soluzioni peggiori, mentre Hoffmann impiega 0,3 s.*
- *A 200 task bimodali la differenza non è significativa (Wilcoxon p = 0,47).*
- *Hoffmann da solo dimostra l'ottimo (soluzione = lower bound) in 110 istanze che i PLI non chiudono.*

# Perché i PLI non chiudono




| Tempi e stazioni | Gap miglior PLI | Gap Hoffmann | Hoffmann raggiunge miglior LB* |
|---|---|---|---|
| **Uniforme, tante** | 2,3% | 2,4% | 11 su 165 |
| **Uniforme, medie** | 0,6% | 0,1% | 67 su 92 |
| **Uniforme, poche** | 0,2% | 0,03% | 13 su 15 |
| **Bimodale, tante** | 1,8% | 2,1% | 1 su 26 |
| **Bimodale, medie** | 0,6% | 0,3% | 19 su 32 |

- ***Poche/medie stazioni (tempi uniformi):*** *Hoffmann raggiunge il lower bound in 80 istanze su 107, quindi lì il bound era già l'ottimo. Al PLI manca il tempo per trovare la soluzione.*
- ***Tante stazioni:*** *soluzioni vicine (circa 2%), ma il bound è debole. Sulle istanze uniformi con ottimo noto, l'ottimo lo supera nell'81% dei casi, contro il 26–34% con meno stazioni.*

** = In 110 dei 111 casi lo dimostra da solo; nell'altro il bound è quello del PLI*

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

# Conclusioni

Osservazioni, Riferimenti

# Osservazioni

- **Fino a 20 task (fino a 100 con tempi bimodali) i PLI chiudono quasi tutto**: sono la scelta giusta. Con tempi uniformi a 50 task si scende già all'84%
- **Con tante stazioni (circa 3 task ciascuna) l'impacchettamento è rigido**: PLI e Hoffmann restano a circa il 2% dal bound e raramente lo raggiungono. Con tempi uniformi il limite è probabilmente il lower bound.
- **RC** dimostra più ottimi con precedenze dense; **PA** è competitivo quando le precedenze sono sparse.
- **Hoffmann conviene quando i PLI non chiudono:** a 200 task uniformi è migliore in 139 istanze su 225, in 0,3 s contro circa 160 s dei PLI. In tutta la campagna dimostra l'ottimo in 110 istanze che i PLI lasciano aperte
- **Il criterio di riempimento conta**: Hoffmann supera Gruppi in 1504 istanze su 1800.
- **La distribuzione dei tempi conta.** Con tempi bimodali le istanze sono più facili: con tante stazioni il lower bound coincide con il task più lungo nel 75% dei casi, e con meno stazioni i task piccoli riempiono gli spazi.
- **Più tempo non basta:** Aumentare il tempo limite serve poco: tra 120 e 180 s si chiudono solo 8 istanze uniformi e 18 bimodali in più.

# Riferimenti e uso nel progetto

| | | |
|---|---|---|
| ***1959 McNaughton:*** | Bound del carico medio | *Lower Bound LC1* |
| ***1963 Hoffmann:*** | Euristica di saturazione delle stazioni | *Euristica di Hoffmann, adattata al SALBP-2* |
| ***1975 Patterson & Albracht:*** | PLI con precedenze per numero di stazione | *Modello PA2-2 e finestre di stazione* |
| ***1988 Johnson:*** | Teste e code ricorsive (FABLE) | *Lower bound LC3, seconda fase (semplificata)* |
| ***1996 Klein & Scholl:*** | Branch & bound per il SALBP-2, bound sui task lunghi | *Lower bound LC2* |
| ***2006 Scholl & Becker:*** | Rassegna di metodi esatti ed euristici | *Schema dei bound LC1-LC3 e ricerca su c delle euristiche* |
| ***2013 Otto, Otto & Scholl:*** | Generatore SALBPGen e disegno degli esperimenti | *Livelli di n e OS, 25 istanze/gruppo, tempi bimodali, Wilcoxon* |
| ***2018 Ritt & Costa:*** | Formulazioni PLI migliorate | *Modello NF4-2* |
| ***2023 Álvarez-Miranda et al.:*** | La difficoltà viene dall'impacchettamento dei task | *Numero di stazioni come fattore di difficoltà* |



TOR VERGATA

UNIVERSITÀ DEGLI STUDI DI ROMA